

AUDIO SYNTHESIZER INVERSION IN SYMMETRIC PARAMETER SPACES WITH APPROXIMATELY EQUIVARIANT FLOW MATCHING (SUPPLEMENTARY MATERIAL)

Anonymous Authors
Anonymous Affiliations
anonymous@ismir.net

1. AUXILIARY DERIVATIONS & PROOFS

1.1 Orbital factorization of $p(\mathbf{x} \mid \mathbf{y})$

Recall that $\mathcal{P} \subset \mathbb{R}^k$ is the space of synthesizer parameters, acted on by $\mathcal{G}$. The $\mathcal{G}$ -orbit of a point $\mathbf{x} \in \mathcal{P}$ is the set of all points in $\mathcal{P}$ that are reachable from $\mathbf{x}$ by actions $g \in \mathcal{G}$. That is:

$$O_{\mathbf{x}} = \{g \cdot \mathbf{x} : g \in \mathcal{G}\}. \quad (1)$$

It is well known that the orbits of a group action on a set form a disjoint partition of the set, which we denote $\mathcal{P} / \mathcal{G}$. We select a representative in $\mathcal{P}$ for each orbit, $O \longleftrightarrow r_O$, which by the partitioning of $\mathcal{P}$ is unique.

Then for any $\mathbf{x} \in \mathcal{P}$ there exists a $g \in \mathcal{G}$ such that $\mathbf{x} = g \cdot r_O$ for some unique O . The uniqueness of g is a slightly more nuanced matter.

1.1.1 Uniqueness of g

We start with a brief definition. The stabilizer subgroup of $\mathcal{G}$ with respect to $\mathbf{x}$ is given by:

$$\text{Stab}(\mathbf{x}) = \{g \in \mathcal{G} : g \cdot \mathbf{x} = \mathbf{x}\}. \quad (2)$$

We say $\mathcal{G}$ is *free* if and only if $\text{Stab}(\mathbf{x}) = \{e_{\mathcal{G}}\}$ for all $\mathbf{x} \in \mathcal{P}$ where $e_{\mathcal{G}}$ is the identity element of $\mathcal{G}$.

Formally, in our setting, we can not safely assume that $\mathcal{G}$ is free as there may exist parameter configurations with non-trivial stabilizers. For example, if two oscillators in a multi-oscillator synthesizer have matching parameters and $\mathcal{G}$ permutes oscillators, then the permutation which swaps them is clearly a member of the stabilizer subgroup. Practically, this is of little concern as the probability of sampling a parameter vector with a non-trivial stabilizer is very low. Nonetheless, we offer the complete derivation addressing non-trivial stabilizers here.

1.2 Factorization of $p(\mathbf{x} \mid \mathbf{y})$

We wish to show that $p(\mathbf{x} \mid \mathbf{y})$ can be decomposed $p(O \mid \mathbf{y})p(g \mid O, \mathbf{y})\eta(O)$. We do so assuming $\mathcal{G}$ is a finite group. First, by the law of total probability we have:

$$p(\mathbf{x} \mid \mathbf{y}) = \sum_{O \in \mathcal{P} / \mathcal{G}} p(\mathbf{x}, O \mid \mathbf{y}) \quad (3)$$

$$= p(\mathbf{x}, O_{\mathbf{x}} \mid \mathbf{y}). \quad (4)$$

Applying the law again, and using the representation of $\mathbf{x}$ as $g \cdot r_{O_{\mathbf{x}}}$, we get:

$$p(\mathbf{x}, O_{\mathbf{x}} \mid \mathbf{y}) = \sum_{g \in \mathcal{G}} p(\mathbf{x}, O_{\mathbf{x}}, g \mid \mathbf{y}) \quad (5)$$

$$= \sum_{g \in \mathcal{G}} p(O_{\mathbf{x}}, g \mid \mathbf{y}) \cdot \delta_{\mathbf{x}, g \cdot r_{O_{\mathbf{x}}}}, \quad (6)$$

where $\delta_{a,b} = 1$ if $a = b$ and 0 otherwise, i.e. the Kronecker delta. By the orbit-stabilizer theorem, there are $|\text{Stab}(r_{O_{\mathbf{x}}})|$ unique elements in $\mathcal{G}$ that satisfy $\mathbf{x} = g \cdot r_{O_{\mathbf{x}}}$. Thus we can state that:

$$p(\mathbf{x} \mid \mathbf{y}) = |\text{Stab}(r_{O_{\mathbf{x}}})| \cdot p(O_{\mathbf{x}}, g \mid \mathbf{y}) \quad (7)$$

$$= p(O_{\mathbf{x}} \mid \mathbf{y}) \cdot p(g \mid O_{\mathbf{x}}, \mathbf{y}) \cdot \eta(O_{\mathbf{x}}), \quad (8)$$

where $\eta(O_{\mathbf{x}}) = |\text{Stab}(r_{O_{\mathbf{x}}})|$. For trivial stabilizers, $\eta(O_{\mathbf{x}}) = 1$. For convenience, in the manuscript we adopt the convention that $O = O_{\mathbf{x}}$.

1.3 Uniformity of $p(g \mid O, \mathbf{y})$

In the manuscript, we assert that if $p(\mathbf{y} \mid \mathbf{x})$ and $p(\mathbf{x})$ are $\mathcal{G}$ -invariant, then $p(g \mid O, \mathbf{y})$ is uniformly distributed over $\mathcal{G}$. Our assumptions are:

$$\begin{aligned} p(\mathbf{y} \mid \mathbf{x}) &= p(\mathbf{y} \mid g \cdot \mathbf{x}) & \forall g \in \mathcal{G} \\ p(\mathbf{x}) &= p(g \cdot \mathbf{x}) & \forall g \in \mathcal{G}. \end{aligned} \quad (9)$$

Then, it is straightforward to show that the posterior is also $\mathcal{G}$ -invariant:

$$p(g \cdot \mathbf{x} \mid \mathbf{y}) \propto p(\mathbf{y} \mid g \cdot \mathbf{x})p(g \cdot \mathbf{x}) \quad (10)$$

$$= p(\mathbf{y} \mid \mathbf{x})p(\mathbf{x}) \quad (11)$$

$$\propto p(\mathbf{x} \mid \mathbf{y}). \quad (12)$$

Recalling our factorization of $p(\mathbf{x} \mid \mathbf{y})$ and using the invariance of the posterior, we observe that for all $h \in \mathcal{G}$:



$$p(O | \mathbf{y})p(g | O, \mathbf{y}) = p(O | \mathbf{y})p(h \cdot g | O, \mathbf{y}), \quad (13)$$

noting that $\eta(O) \geq 1$. Therefore,

$$p(g | O, \mathbf{y}) = p(h \cdot g | O, \mathbf{y}), \quad \forall h \in \mathcal{G}. \quad (14)$$

Of course, this is only true if $p(g | O, \mathbf{y})$ is uniform over $\mathcal{G}$, and so we can state, as in the manuscript, that:

$$p(\mathbf{x} | \mathbf{y}) \propto p(O | \mathbf{y})\eta(O) \quad (15)$$

1.4 Conditional symmetry in PARAM2TOK

Definition 1.1 (PARAM2TOK). Let $P2T : \mathcal{P} \rightarrow \mathbb{R}^{n \times d}$ be the mapping from parameters to tokens:

$$P2T(x) = \mathbf{A}h_\theta(\text{diag}(\mathbf{x}) \cdot \mathbf{Z}),$$

for $\mathbf{A} \in \mathbb{R}^{n \times k}$, $\mathbf{Z} \in \mathbb{R}^{k \times d}$, and FFN h_θ applied row-wise.

We aim to show that, in combination with the first Transformer layer, the PARAM2TOK block is capable of representing a conditional symmetry by implementing conditional equivariance. That is, the equivariance can be broken depending on the value of some entry in the parameter vector $\mathbf{x} \in \mathcal{P}$. We do so by construction, simply to illustrate that this is possible.

For simplicity, we assume that the residual multi-head attention block implements the identity transformation, which is possible if the output projection is zero. We also ignore the effect of layer normalization, without loss of generality. Thus, we focus on the effect of the FFN in the first transformer layer.

Let $\mathbf{x} = [a \ b \ c]^T \in [0, 1]^3$ be our parameter vector.

Let the permutation $\pi(\mathbf{x}) = [b \ a \ c]^T$. Let the permutation ρ act on a $2 \times d$ matrix by swapping its rows, i.e.

$$\rho \begin{pmatrix} \mathbf{a} & \mathbf{b} \end{pmatrix}^T = \begin{pmatrix} \mathbf{b} & \mathbf{a} \end{pmatrix}^T.$$

Let the FFN be defined $\text{FFN}(\mathbf{y}) = \text{ReLU}(\mathbf{y} \cdot \mathbf{W}_{\text{in}}) \cdot \mathbf{W}_{\text{out}}$.

Theorem 1. There exist P2T and FFN parameters such that

$$\begin{aligned} c = 1 : & \text{FFN} \circ P2T \circ \pi(\mathbf{x}) = \rho \circ \text{FFN} \circ P2T(\mathbf{x}) \\ c = 0 : & \text{FFN} \circ P2T \circ \pi(\mathbf{x}) \neq \rho \circ \text{FFN} \circ P2T(\mathbf{x}). \end{aligned}$$

Proof. First we construct our P2T parameters. We set the parameter embeddings $\mathbf{Z}$ as follows:

$$\mathbf{Z} = \begin{bmatrix} u & v_1 & 0 \\ u & v_2 & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

for $u, v_1, v_2 \in [0, 1]$. Note the assumption that $a, b, c, u, v_1, v_2 \in [0, 1]$ is made without loss of generality for ease of construction. In practice, values on intervals that include negative values can be shifted to the positive real line by bias terms.

We set h_θ such that its weights are identity matrices and assume a ReLU activation. Thus, because we have enforced positivity $h_\theta(\text{diag}(\mathbf{x}) \cdot \mathbf{Z}) = \text{diag}(\mathbf{x}) \cdot \mathbf{Z}$.

We then construct the assignment matrix $\mathbf{A}$:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}.$$

Then, finally, we have:

$$\mathbf{X}_0 = P2T(\mathbf{x}) = \begin{bmatrix} au & av_1 & c \\ bu & bv_2 & c \end{bmatrix}.$$

Let $\mathbf{W}_{\text{out}} = I_3$. Let

$$\mathbf{W}_{\text{in}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -1 & 0 \end{bmatrix}.$$

Then

$$\mathbf{X}_0 \cdot \mathbf{W}_{\text{in}} = \begin{bmatrix} au & av_1 - c & 0 \\ bu & bv_2 - c & 0 \end{bmatrix}.$$

Then for $c = 1$ we have:

$$\mathbf{Y}_1 = \text{FFN}(\mathbf{X}_0)|_{c=1} = \begin{bmatrix} au & 0 & 0 \\ bu & 0 & 0 \end{bmatrix}$$

and for $c = 0$ we have:

$$\mathbf{Y}_0 = \text{FFN}(\mathbf{X}_0)|_{c=0} = \begin{bmatrix} au & av_1 & 0 \\ bu & bv_2 & 0 \end{bmatrix}.$$

We now apply the permutation operators π and ρ to test our construction.

$$\rho(\mathbf{Y}_1) = \begin{bmatrix} bu & 0 & 0 \\ au & 0 & 0 \end{bmatrix}$$

while

$$\begin{aligned} \text{FFN} \circ P2T \circ \pi(\mathbf{x})|_{c=1} &= \text{FFN} \left(\begin{bmatrix} bu & bv_1 & 1 \\ au & av_2 & 1 \end{bmatrix} \right) \\ &= \begin{bmatrix} bu & 0 & 0 \\ au & 0 & 0 \end{bmatrix}. \end{aligned}$$

Meaning $\text{FFN} \circ P2T \circ \pi(\mathbf{x}) = \rho \circ \text{FFN} \circ P2T(\mathbf{x})$ and we have satisfied the $c = 1$ case.

Meanwhile, for $c = 0$:

$$\rho(\mathbf{Y}_0) = \begin{bmatrix} bu & bv_2 & 0 \\ au & av_1 & 0 \end{bmatrix}$$

$$\begin{aligned} \text{FFN} \circ P2T \circ \pi(\mathbf{x})|_{c=0} &= \text{FFN} \left(\begin{bmatrix} bu & bv_1 & 0 \\ au & av_2 & 0 \end{bmatrix} \right) \\ &= \begin{bmatrix} bu & bv_1 & 0 \\ au & av_2 & 0 \end{bmatrix}. \end{aligned}$$

Clearly, $bv_2 \neq bv_1$ and $av_2 \neq av_1$, and so we have satisfied that $\text{FFN} \circ P2T \circ \pi(\mathbf{x}) \neq \rho \circ \text{FFN} \circ P2T(\mathbf{x})$ when $c = 0$. We have therefore constructed a conditional equivariance and we are done. $\square$

1.5 proof by construction that param2tok can be approximately equivariant

1.6 Transferability to DDSP

1.7 Parameter metrics under symmetry

Assume f is continuous and non-constant.

Specifically, for a point x in parameter space and an arbitrary non-trivial symmetry $g \cdot x$ (i.e. $f(x) = f(g \cdot x)$ and $x \neq g \cdot x$), we can always construct an open ball $B_x = \{x' : \|x' - x\|_2 < \|g \cdot x - x\|_2\}$ such that $\|f(x') - f(x)\|_2 \geq \|f(g \cdot x) - f(x)\|_2$ for all $x' \in B_x$. In other words, for any given pair of points related by symmetry, we can always find points with a lower $\mathcal{L}_2$ parameter distance and a higher $\mathcal{L}_2$ signal distance.¹

1.8 VAE + RealNVP parameter regression collapses to average rather than learning multimodal distribution.

2. MODELS, HYPERPARAMETERS, AND TRAINING DETAILS

2.1 k -osc Models

All models were trained for 400k steps using the Adam optimizer with a learning rate of 10^{-4} . All other optimizer parameters were set to their PyTorch defaults. Training was conducted with BFloat16 mixed precision. Gradient norms were clipped to a maximum value of 1.0.

2.1.1 CNN Encoder

All models used in the k -osc experiment use the same CNN architecture to extract a representation of the synthesizer signal. This is illustrated in Fig. 1.

The encoder starts by computing the discrete Fourier transform of the signal and taking its complex modulus. There is then a stack of 4 residual CNN blocks, which are interspersed by a strided convolution which downsamples by a factor of 3 while doubling the channel dimension. The dimension of the first block is 24. The final feature map is flattened and passed to a linear layer, and then a residual MLP block, producing a 128 dimensional vector which is passed to the rest of the network.

2.1.2 CNF models

These models are illustrated in Fig. 1(a). CNF (PARAM2TOK) and CNF (Equivariant) both use a diffusion transformer (DiT) vector field network with 5 layers. Each layer contains two residual blocks. The first consists of layer normalization followed by a multi-head self attention module with 4 heads and a token dimension of 128. The second consists of a further layer normalization followed by a feed-forward network (FFN) with an input/output dimension of 128, a hidden dimension of 256, and a GELU activation.

CNF (MLP) replaces the DiT with a residual multi-layer perceptron (MLP) constructed by simply omitting the attention residual blocks leaving only residual FFN layers.

¹ For a full proof of this proposition for $\mathcal{L}_2$ and $\mathcal{L}_1$ distances, see the supplementary material.

Note that the MLP is applied across the full parameter vector rather than tokenwise — that is, it does not have an intrinsic equivariance to permutation. We use a model dimension of 192 and a total of 7 layers.

2.1.2.1 Input projection

CNF (Equivariant) uses the natural tokenization of parameters implied by the synthesizer’s structure. That is, one oscillator’s parameters are assigned to each token. These 3-dimensional vectors are then projected up to the model dimension by a linear layer. This naturally yields k internal model tokens.

CNF (PARAM2TOK) uses the PARAM2TOK projection, where the number of model tokens is a tunable hyperparameter corresponding to the “available” degree of symmetry. While in future work we intend to study the effect over under- and over-specifying this parameter, here we are focused simply on the ability of the model to discover the symmetry, and so we simply set it to k , assuming this is known *a priori*.

CNF (MLP) simply projects the parameter vector up to the model dimension with a linear layer.

2.1.2.2 Conditioning

There are two sources of conditioning: the CNN’s output, and the flow matching time step. The latter is represented as a simple scalar value, which is concatenated to the CNN’s output, giving a 129 dimensional conditioning vector.

We use adaptive layer normalization (Ada-LN) to introduce conditioning information to the network. This means that after each layer normalization, a pointwise affine transformation is applied, with shift and scale parameters produced by a separate FFN per layer applied to a conditioning vector. The output of the non-residual pathway of each residual block is also scaled by a further parameter produced by the conditioning FFN. For CNF (PARAM2TOK) and CNF (Equivariant), this means the layer-wise conditioning FFN has a total output dimension of $6 \times 128 = 768$, which is split to produce the affine parameters. For CNF (MLP) it outputs a $3 \times 128 = 384$ dimensional vector as there is only one residual block per layer. Before being passed to each layer, the conditioning vector is also passed through a shared FFN. Note that, unlike prior work, we find that an Ada-LN-Zero initialization (where the conditioning FFN is initialized to leave the main network’s activations unchanged) actually harms performance, and so we do not use it here.

With a probability of 10%, we replace the CNN conditioning vector with a learnt “dropout” vector. Doing so allows us to use classifier-free guidance [1, 2] at inference, which we apply with a scale of 2.0, determined through experimentation to provide reasonable performance across models. Future work will offer a more comprehensive study of these design parameters.

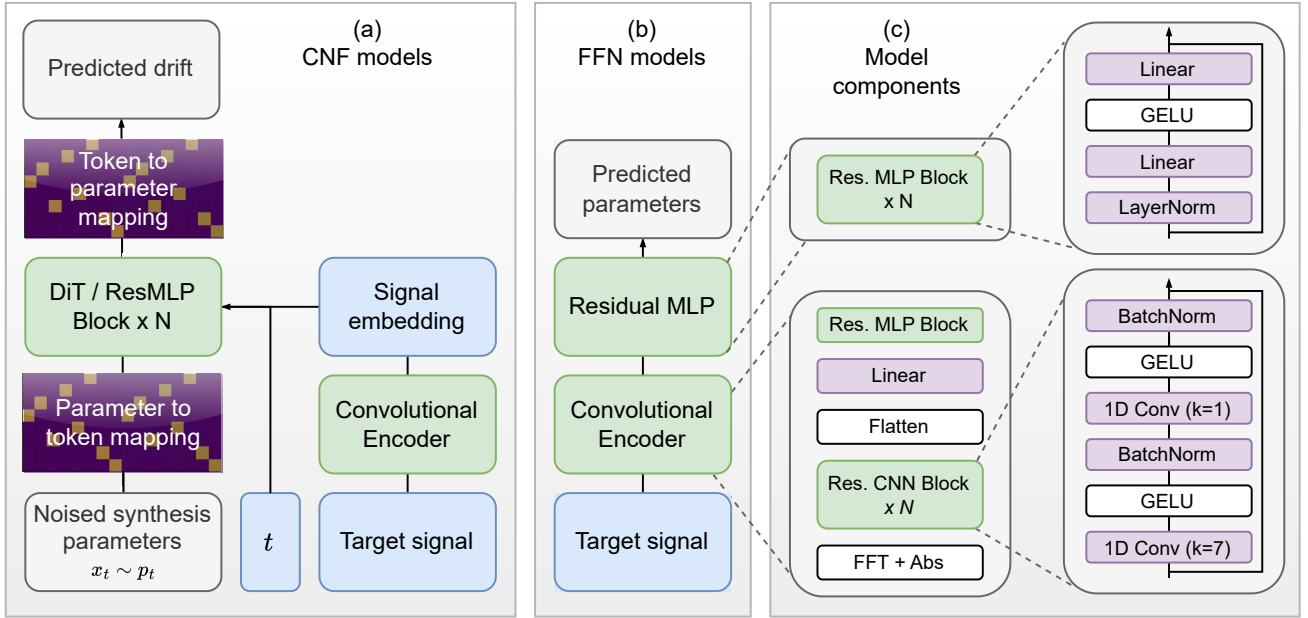


Figure 1. Diagram of models used in the k -osc task.

2.1.3 Regression models

These are illustrated in Fig. 1(b). All use the same architecture and vary only in their loss computation.

The output of the CNN is passed directly to a residual MLP. Each residual MLP block consists, again, of a residual block containing a layer normalization, followed by a linear layer, a GELU activation, and a final linear layer before adding the residual signal. The residual MLP head contains of 4 layers with a width of 384. A final linear layer is used to project the MLP’s output back to the dimension of the parameter vector.

2.1.3.1 Loss functions

FFN (MSE) simply regresses parameters using a mean-squared error loss. This means the model does not account for symmetry at either the architectural or loss level.

FFN (Sort) adopts the sorting strategy for symmetry breaking adopted in prior sound matching [3] and EQ matching [4] work. In this approach, the target parameter vector is modified by sorting its entries according to oscillator frequency. In this sense, the symmetry of the problem with respect to any individual parameter vector is effectively broken — there exists a canonical, sorted form. However, across the space of all possible parameter vectors, prior theoretical work [5] tells us that this does *not* resolve the responsibility problem, as similar *sets* of parameters can be selected that lead to big differences between parameter *vectors*. We thus expect improved performance over the FFN (MSE) baseline — there will no longer be a risk of conflicting gradients with respect to any one example — but degraded performance compared to models which account for the symmetry at an architectural level, as the responsibility problem still exists. We do not include the FFN (Sort) variant on the asymmetric version of

the task, as targets are effectively already implicitly sorted.

We use FFN (Chamfer) to study the effect of permutation invariant loss functions. Like with FFN (Sort), we expect this to improve over FFN (MSE) by reducing conflicts in the learning signal, but to still suffer degraded performance due to the responsibility problem. We use a differentiable Chamfer distance [6] as our loss function, defined between two sets $\mathcal{X}$ and $\mathcal{Y}$ as:

$$\mathcal{L}_{CD}(\mathcal{X}, \mathcal{Y}) = \sum_{\mathbf{y} \in \mathcal{Y}} \min_{\mathbf{x} \in \mathcal{X}} d(\mathbf{x}, \mathbf{y}) + \sum_{\mathbf{x} \in \mathcal{X}} \min_{\mathbf{y} \in \mathcal{Y}} d(\mathbf{x}, \mathbf{y}). \quad (16)$$

Here, we adopt the same mapping to set elements as CNF (Equivariant) and define d as the mean squared error. Losses based on the Chamfer distance are commonly used as an approximation to the true linear assignment distance, which requires online solution of the corresponding combinatorial optimization problem. A Chamfer-style loss was used by Engel, et al. [7] to match the frequencies of sets of sinusoids.

2.2 Surge XT models

We train all models for 1 million steps. For all models except VAE + RealNVP, we use the Adam optimizer, with an initial learning rate of 10^{-4} , which is decayed to 10^{-6} using the cosine learning rate schedule. For VAE + RealNVP, we adopt an initial learning rate of 2×10^{-4} . Again, we train under BFloat16 mixed precision, and clip gradient norms, though on this task we found a maximum norm of 0.5 to better ensure stability.

2.2.1 CNF models

The vector field architecture of the CNF (PARAM2TOK) and CNF (MLP) models is effectively equivalent to their

age" between subsystems. We thus adopt a $\mathcal{L}_1$ penalty on the entries in the $\mathbf{A}$ matrix, which we weight by a factor of 0.01.

While these initialization and regularization strategies were determined through a combination of problem insight and preliminary experimentation, we do find them necessary to achieve good performance. We intend, in our next publication, to include detailed ablations and parameter sweeps to draw better insight into their behaviour.

3. METRICS

Here we provide full details of metrics used in the k -osc and Surge XT experiments.

3.1 k -osc metrics

LAC: The linear assignment cost is defined between two sets of points by finding the minimum cost attainable by assignments between them. This is achieved by solving the linear assignment problem, typically using the Hungarian or Jonker-Volgenant algorithms. For ground truth parameters $\mathbf{x}$ and predicted parameters $\hat{\mathbf{x}}$, and permutations of oscillators $\pi \in S_k$, the cost is defined:

$$\text{LAC} = \frac{1}{3k} \min_{\pi \in S_k} \|\mathbf{x} - \pi \hat{\mathbf{x}}\|_2^2. \quad (17)$$

The resulting metric is thus a permutation invariant distance between two sets of parameters, because regardless of how parameters are permuted, the minimum cost remains unchanged.

MSE: On the asymmetric variant of k -osc, the LAC would be an inappropriate metric as it would reward incorrect predictions. We thus use the straightforward mean squared error between the predicted and ground truth parameter vectors in its place.

LSD: The log-spectral distance measures the pointwise dissimilarity between the log-scaled magnitude spectra of two signals. Here, given ground truth signal $\mathbf{y}$ and predicted signal $\hat{\mathbf{y}}$, we compute the discrete Fourier transforms $\mathbf{Y}$ and $\hat{\mathbf{Y}}$. Then the LSD is given by:

$$\text{LSD} = \sqrt{\frac{1}{N} \sum_{n=1}^N (\log |Y_n| - \log |\hat{Y}_n|)^2}. \quad (18)$$

3.2 Surge XT metrics

MSS: Multi-scale time-frequency representations have found common use as both training loss functions and metrics for audio similarity across a variety of audio processing tasks. They typically involve multiple evaluations of a time-frequency transform, such as the STFT, with differing filter and stride lengths, selected to achieve sensitivity to spectrotemporal structures at differing scales. Effectively, they allow heuristic compensation for the intrinsic trade-off between time and frequency resolution in designing a transform.

Here, we use log-scaled Mel spectrograms with window lengths of 10, 25, and 100 milliseconds, and corresponding

hop lengths of 5, 10, and 50ms. We use filterbanks with 32, 64, and 128 mel bands, respectively. We compute the mean absolute error between corresponding spectrograms, and aggregate these by averaging across scales.

wMFCC: Due to the intricate interactions between synthesizer parameters, it is possible for a slight error in one parameter to shift the synthesizer's output in time or frequency, thus leading to simple spectrotemporal metrics unfairly penalising an otherwise good prediction. Even a few milliseconds error on an amplitude envelope, or a few cents on an oscillator frequency, could cause a signal which is almost indistinguishable perceptually from the ground truth to suffer a poor MSS value.

Thus, we include a *warped Mel-frequency cepstral coefficient* metric as a more "malleable" alternative to straightforward spectral distances. The metric is computed in two steps. First, the first 20 MFCCs of both the ground truth and predicted signals are computed, with a hop length of 10ms. As MFCCs are more sensitive to the spectral envelope than the absolute pitch of a signal, this allows for greater robustness to small pitch errors. Secondly, the dynamic time warping algorithm is applied to the two MFCC series using the L1 distance, allowing for small timing deviations to be accommodated. The cost of the optimal DTW path is then taken as the wMFCC value.

SOT: While MSS and wMFCC are sensitive to fine-grained spectral similarity and overall timbral evolution, respectively, they do not provide good intuition about the distance "along" the frequency axis. For example, consider a ground truth signal containing a single frequency component at 100Hz, and two predictions, one at 500Hz and one at 5kHz. Ignoring artifacts due to spectral leakage, these predictions would yield equal MSS distances from the ground truth, as the L1 distance is effectively a pointwise comparison. While previous works on musical audio synthesis [14, 15] have employed fundamental frequency estimators such as CREPE [16] to compute a pitch similarity metric, we believe this would be unsuitable for the audio in our dataset, which is frequently aharmonic, aperiodic, or exhibits characteristics well outside the training distribution of common pitch estimators.

We thus adopt the *spectral optimal transport* (SOT) distance, first proposed as a differentiable loss function for optimizing frequency parameters [17], as a more permissive metric for similarity along the frequency axis. The SOT distance follows the intuition that a greater difference between two frequencies should correspond to a larger distance. It is computed as the Wasserstein-1 distance between normalized magnitude spectra:

$$\mathcal{W}_1(\mu, \nu) = \inf_{\pi \in \Gamma(\mu, \nu)} \int |x - y| d\pi(x, y), \quad (19)$$

where μ and ν are probability measures, here defined as a sum of Dirac measures located at Fourier harmonic frequencies and weighted by their magnitudes in the ground truth and predicted spectra respectively, and Γ is the set of all couplings between elements in μ and ν . The infimum thus corresponds to the optimal transport cost be-

465 tween the two measures. In this one-dimensional case, a
466 simple closed form solution is available.

467 3.2.0.1 RMS

468 Finally, to compare the overall amplitude evolution of
469 the predicted signal to the ground truth, we compute the
470 framewise root mean squared energy envelopes of the two
471 signals and take their cosine similarity. In this way, we
472 are able to compare the manner in which signal amplitude
473 evolves over time while remaining invariant to overall am-
474 plitude scaling errors.

475 4. LEARNT PARAM2TOK PARAMETERS

476 5. DATA

477 5.1 k -osc dataset parameters

478 5.2 Surge XT dataset

479 Surge XT is described as a “hybrid” synthesizer, which in-
480 corporates multiple different methods for producing sound,
481 many of which are capable of introducing non-trivial per-
482 mutation symmetries.

483 The synthesizer consists of multiple “scenes”, each of
484 which contains a copy of the synthesizer’s sound genera-
485 tion engine. The outputs of each scene can be routed to a
486 mixer for aggregation and further processing, while MIDI
487 inputs to each scene’s sound engine can be determined by
488 different voice split and layering rules. For the present ex-
489 periment, we disable the second scene and focus on just
490 one instance of the sound engine.

491 Due to the complexity of the audio effects module in
492 Surge XT, and the wide variety of effect types², we
493 choose to adopt a fixed effect routing for our experiments,
494 using only the Chorus, Delay, and Reverb II effects.

495 Similarly, due to the flexibility of Surge XT’s modu-
496 lation routing and large number of multi-functional mod-
497 ulation sources, we adopt a fixed routing which reflects
498 common usage of LFOs and auxiliary envelopes. Specif-
499 ically, we map LFOs 1 and 2 to the cutoff frequencies of
500 the two filters, and LFOs 3-5 to the pulse width param-
501 eters of the three oscillators. We expose a limited subset of
502 the mode parameters of each of these modulation sources,
503 corresponding to a selection of different LFO and noise
504 sources, and excluding envelope generators, sequencers,
505 and programmable function generators. We fix LFO 6 to
506 the attack-decay-sustain-release (ADSR) envelope genera-
507 tor mode and map it to the global pitch modulation. We
508 thus refer to it as “Pitch EG”, and its amplitude as “Pitch
509 EG Mod Amount”.

510 In general, we endeavoured to retain as many synthesis
511 parameters as possible in the prediction task, deactivating
512 only those which introduced an unnecessary redundancy
513 (e.g. global volume and scene volume are equivalent in
514 our configuration) or which are irrelevant to our particu-
515 lar experimental setup (e.g. filter keytracking, LFO trigger
516 mode, pitch bend amount, etc.) Details of the parameters

517 used in both the Simple and Full versions of the Surge XT
518 task are provided in Table 1.

519 All parameters, including discrete values, are passed
520 to the VST as floating point values on the interval $[0, 1]$.
521 These values are then internally scaled and quantized by
522 the plugin. For most continuous parameters we simply
523 uniformly sample from the unit interval. For most dis-
524 crete parameters, we sample uniformly from the possible
525 values. However, in certain cases we increase the proba-
526 bility of selecting a particular value, where it is reasonable
527 to expect that this value is the usual or “default” setting for
528 this parameter. For example, each LFO’s amplitude has
529 50% probability of being 0, and will otherwise be sampled
530 from the unit interval. Similarly, the waveshaper type is
531 set to “None” with probability 50%, and otherwise sam-
532 pled uniformly from the list from remaining types. We
533 also truncate the sampling interval for certain parameter
534 ranges where extreme values could cause issues with clip-
535 ping (e.g. the upper bound for certain gain parameters is as
536 high as +48dB), or where certain values do not make sense
537 in our experimental setup (e.g. an amplitude envelope with
538 an attack time of length 30s is unlikely to produce a usable
539 sound given the length of our MIDI note event). Rather
540 than listing all sampling distributions here, we instead re-
541 fer the reader to our source code repository for full details:
542 [github URL](#).

543 While previous sound matching work typically uses
544 fixed MIDI note pitches and/or durations, here we opt to
545 view these as predictable parameters in order to increase
546 the adaptability of our system to a variety of different in-
547 puts. We thus uniformly sample MIDI pitches from a two
548 octave interval centred at C4 (MIDI note 60) and uniformly
549 sample note-on and note-off event times within the 4 sec-
550 ond window.

² Surge XT even contains a neural network as an audio effect!

	Parameter	Simple	Full	Notes
2×	Filter i Cutoff	✓	✓	Options include low-, band-, and high-pass filters at 12dB and 24dB per octave, as well as notch, comb, and allpass filters.
	Filter i FEG Mod. Amount	✓	✓	
	Filter i Resonance	✓	✓	
	Filter i Type	✗	✓	
5×	LFO j Amplitude	✓	✓	Continuously warps the shape of the LFO.
	LFO j Attack	✓	✓	
	LFO j Decay	✓	✓	
	LFO j Deform	✓	✓	
	LFO j Hold	✓	✓	
	LFO j Phase	✓	✓	
	LFO j Rate	✓	✓	
	LFO j Release	✓	✓	
	LFO j Sustain	✓	✓	
	LFO j Type	✗	✓	
3×	Osc. k Mute	✗	✓	Oscillator signal patch point.
	Osc. k Octave	✗	✓	
	Osc. k Pitch	✓	✓	
	Osc. k Pulse	✓	✓	
	Osc. k Route	✗	✓	
	Osc. k Sawtooth	✓	✓	
	Osc. k Sync	✓	✓	
	Osc. k Triangle	✓	✓	
	Osc. k Unison Detune	✗	✓	
	Osc. k Unison Voices	✗	✓	
	Osc. k Volume	✓	✓	Options are Digital and Analogue. These change the curvature of envelope segments.
	Osc. k Width	✓	✓	
	Amp. EG Attack	✓	✓	
	Amp. EG Decay	✓	✓	
	Amp. EG Env. Mode	✗	✓	
	Amp. EG Release	✓	✓	
	Amp. EG Sustain	✓	✓	
	Feedback	✗	✓	
	Filter Balance	✗	✓	
	Filter Routing	✗	✓	
	Filter EG Attack	✓	✓	Same options as Amp. EG Env. Mode.
	Filter EG Decay	✓	✓	
	Filter EG Env. Mode	✗	✓	
	Filter EG Release	✓	✓	
	Filter EG Sustain	✓	✓	
	FM Depth	✗	✓	Behaviour is conditional on FM Routing.
	FM Routing	✗	✓	Allows frequency modulation between the oscillators in different configurations.
	Highpass	✓	✓	As well as the two configurable filters, there is a global high-pass filter.
	Pitch EG Mod. Amount	✓	✓	Pitch EG implemented via LFO 6 set to ADSR mode.
	Pitch EG Attack	✓	✓	
	Pitch EG Decay	✓	✓	
	Pitch EG Deform	✓	✓	
	Pitch EG Hold	✓	✓	
	Pitch EG Release	✓	✓	

Pitch EG Sustain	✓	✓	
Noise Color	✗	✓	
Noise Mute	✗	✓	
Noise Route	✗	✓	Noise signal patch point.
Noise Volume	✓	✓	
Osc. Drift	✗	✓	Independently varies the pitch of all oscillators and their unison voices.
Pan	✗	✓	
Ring Modulation 1x2 Mute	✗	✓	Ring modulation between oscillators is always running with fixed modulation depth, and can be mixed into the signal.
Ring Modulation 1x2 Route	✗	✓	Ring Mod. signal patch point.
Ring Modulation 1x2 Volume	✗	✓	
Ring Modulation 2x3 Mute	✗	✓	
Ring Modulation 2x3 Route	✗	✓	Ring Mod. signal patch point.
Ring Modulation 2x3 Volume	✗	✓	
VCA Gain	✗	✓	
Waveshaper Drive	✗	✓	
Waveshaper Type	✗	✓	Choice between soft, hard, asymmetric, and sine waveshapers.
Width	✗	✓	
Chorus Delay Feedback	✗	✓	
Chorus Delay Time	✗	✓	
Chorus EQ High Cut	✗	✓	
Chorus EQ Low Cut	✗	✓	
Chorus Modulation Depth	✗	✓	
Chorus Modulation Rate	✗	✓	
Chorus Output Mix	✗	✓	
Chorus Output Width	✗	✓	
Delay Time Left	✗	✓	
Delay Time Right	✗	✓	
Delay Feedback EQ Crossfeed	✗	✓	
Delay Feedback EQ Feedback	✗	✓	
Delay Feedback EQ High Cut	✗	✓	
Delay Feedback EQ Low Cut	✗	✓	
Delay Input Channel	✗	✓	
Delay Modulation Depth	✗	✓	
Delay Modulation Rate	✗	✓	
Delay Output Mix	✗	✓	
Delay Output Width	✗	✓	
Reverb EQ HF Damping	✗	✓	
Reverb EQ LF Damping	✗	✓	
Reverb Output Mix	✗	✓	
Reverb Output Width	✗	✓	
Reverb Pre-delay	✗	✓	
Reverb Buildup	✗	✓	
Reverb Decay Time	✗	✓	
Reverb Diffusion	✗	✓	
Reverb Modulation	✗	✓	
Reverb Room Size	✗	✓	
Note Start	✓	✓	MIDI note parameter.
Note End	✓	✓	MIDI note parameter.
Note Pitch	✓	✓	MIDI note parameter.

Table 1: Parameters of the Surge XT synthesizer used in the *Surge Simple* and *Surge Full* datasets.

6. REFERENCES

- [1] Jonathan Ho and Tim Salimans. “Classifier-Free Diffusion Guidance”. In: *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*. Dec. 8, 2021. URL: <https://openreview.net/forum?id=qw8AKxfYbI> (visited on 03/19/2025).
- [2] Qinqing Zheng et al. *Guided Flows for Generative Modeling and Decision Making*. Dec. 7, 2023. DOI: 10.48550/arXiv.2311.13443. arXiv: 2311.13443 [cs]. Pre-published.
- [3] Naotake Masuda and Daisuke Saito. “Improving Semi-Supervised Differentiable Synthesizer Sound Matching for Practical Applications”. In: *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 31 (2023), pp. 863–875. ISSN: 2329-9304. DOI: 10.1109/TASLP.2023.3237161.
- [4] Shahan Nercessian. “Neural Parametric Equalizer Matching Using Differentiable Biquads”. In: *Proceedings of the 23rd International Conference on Digital Audio Effects*. DAFx2020. Vienna, Austria, 2020, p. 8.
- [5] Ben Hayes, Charalampos Saitis, and György Fazekas. “The Responsibility Problem in Neural Networks with Unordered Targets”. In: *The First Tiny Papers Track at ICLR 2023*. ICLR. Kigali, Rwanda, May 5, 2023. URL: <https://openreview.net/pdf?id=jd7Hy1jRiv4>.
- [6] Harry G Barrow et al. “Parametric Correspondence and Chamfer Matching: Two New Techniques for Image Matching”. In: *Proceedings: Image Understanding Workshop*. Science Applications, Inc, 1977, pp. 21–27.
- [7] Jesse Engel, Rigel Swavely, and Adam Roberts. “Self-Supervised Pitch Detection by Inverse Audio Synthesis”. In: *Proceedings of the International Conference on Machine Learning*. ICML 2020. 2020, p. 9.
- [8] Yuan Gong, Yu-An Chung, and James Glass. “AST: Audio Spectrogram Transformer”. In: *Proc. Interspeech 2021*. 2021, pp. 571–575. DOI: 10.21437/Interspeech.2021-698.
- [9] Ruibin Xiong et al. “On Layer Normalization in the Transformer Architecture”. In: *Proceedings of the 37th International Conference on Machine Learning*. International Conference on Machine Learning. PMLR, Nov. 21, 2020, pp. 10524–10533. URL: <https://proceedings.mlr.press/v119/xiong20b.html> (visited on 03/12/2025).
- [10] Fred Bruford, Frederik Blang, and Shahan Nercessian. “Synthesizer Sound Matching Using Audio Spectrogram Transformers”. In: *Proceedings of the 27th International Conference on Digital Audio Effects*. DAFx24. Guildford, Surrey, Sept. 3–7, 2024.
- [11] Gwendal Le Vaillant, Thierry Dutoit, and Sebastien Dekeyser. “Improving Synthesizer Programming From Variational Autoencoders Latent Space”. In: *2021 24th International Conference on Digital Audio Effects (DAFx)*. 2021 24th International Conference on Digital Audio Effects (DAFx). Vienna, Austria: IEEE, Sept. 8, 2021, pp. 276–283. ISBN: 978-3-200-08378-3. DOI: 10.23919/DAFx51585.2021.9768218.
- [12] Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. “Density Estimation Using Real NVP”. In: *International Conference on Learning Representations*. Feb. 6, 2017. URL: <https://openreview.net/forum?id=HkpbH9lx> (visited on 03/18/2025).
- [13] Philippe Esling et al. “Flow Synthesizer: Universal Audio Synthesizer Control with Normalizing Flows”. In: *Applied Sciences* 10.1 (Dec. 2020), p. 302. DOI: 10.3390/app10010302.
- [14] Lamtharn Hantrakul et al. “Fast and Flexible Neural Audio Synthesis”. In: *Proceedings of the 20th International Society for Music Information Retrieval Conference*. ISMIR. Delft, The Netherlands, 2019, pp. 524–530.
- [15] Jesse Engel et al. “DDSP: Differentiable Digital Signal Processing”. In: *8th International Conference on Learning Representations*. ICLR 2020. Addis Ababa, Ethiopia, Apr. 2020.
- [16] Jong Wook Kim et al. “Crepe: A Convolutional Representation for Pitch Estimation”. In: *2018 IEEE International Conference on Acoustics, Speech, and Signal Processing, ICASSP 2018 - Proceedings*. 2018 IEEE International Conference on Acoustics, Speech, and Signal Processing, ICASSP 2018. Institute of Electrical and Electronics Engineers Inc., Sept. 10, 2018, pp. 161–165. DOI: 10.1109/ICASSP.2018.8461329.
- [17] Bernardo Torres, Geoffroy Peeters, and Gaël Richard. “Unsupervised Harmonic Parameter Estimation Using Differentiable DSP and Spectral Optimal Transport”. In: *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). Seoul, Korea, Republic of: IEEE, Apr. 14, 2024, pp. 1176–1180. ISBN: 9798350344851. DOI: 10.1109/ICASSP48485.2024.10447011.